

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 3 | AULA 01 -

CONCEITOS BÁSICOS DE ARQUITETURA DISTRIBUÍDA

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3

HANDS ON 4

SAIBA MAIS 5

O QUE VOCÊ VIU NESTA AULA? 21

REFERÊNCIAS 22

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Você já se perguntou como as grandes empresas, como Netflix, Amazon, Google ou Facebook, conseguem atender milhões de pessoas ao mesmo tempo, em diferentes partes do mundo, sem ficar indisponível ou apresentar problemas? Esse desafio é a base de uma arquitetura distribuída, um dos pilares na decisão de uma arquitetura para construção de um sistema com alta resiliência e escalabilidade.

Entender como esses sistemas funcionam vai muito além de aprender apenas o conceito. Envolve explorar as soluções que fazem a tecnologia funcionar de forma eficiente e resiliente no dia a dia.

Nessa aula, vamos explorar os fundamentos da arquitetura distribuída, explicar sobre os tipos de sistemas, os diferentes estilos de arquiteturas e as estratégias de comunicação que garantem que os serviços sejam rápidos, escaláveis e confiáveis. Prepare-se para explorar exemplos práticos que vão conectar teoria à prática, ajudando você a compreender como essas soluções impactam diretamente a experiência do usuário e a operação das empresas.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

Nesta videoaula, vamos nos aprofundar nos conceitos de Arquitetura Distribuída, explorando diferentes estilos de arquiteturas e suas aplicações práticas. O objetivo é explicar como cada tipo de arquitetura funciona, destacando os contextos em que são mais eficazes e as situações em que devem ser aplicadas.

Acompanhe os vídeos para consolidar o entendimento teórico e prepare-se para visualizar como essas arquiteturas sustentam os sistemas que usamos diariamente!

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

Vamos embarcar na jornada teórica para entender os sistemas distribuídos. Exploraremos seus principais conceitos, como suas arquiteturas funcionam, os tipos de comunicação distribuída e os seus desafios.

Os sistemas distribuídos consistem em um conjunto de computadores independentes que trabalham juntos para atingir um objetivo comum, compartilhando recursos e informações. Essa abordagem resolve limitações de sistemas monolíticos, permitindo escalabilidade, flexibilidade e resiliência.

Sistemas distribuídos são uma abordagem de design em que um sistema é composto por vários componentes interconectados, geralmente localizados em computadores ou servidores diferentes. Não há exigência de proximidade física entre esses computadores, eles podem ser fiscalmente próximos e conectados por uma rede local (LAN) ou conectados por distância utilizando uma rede de internet (WAN).

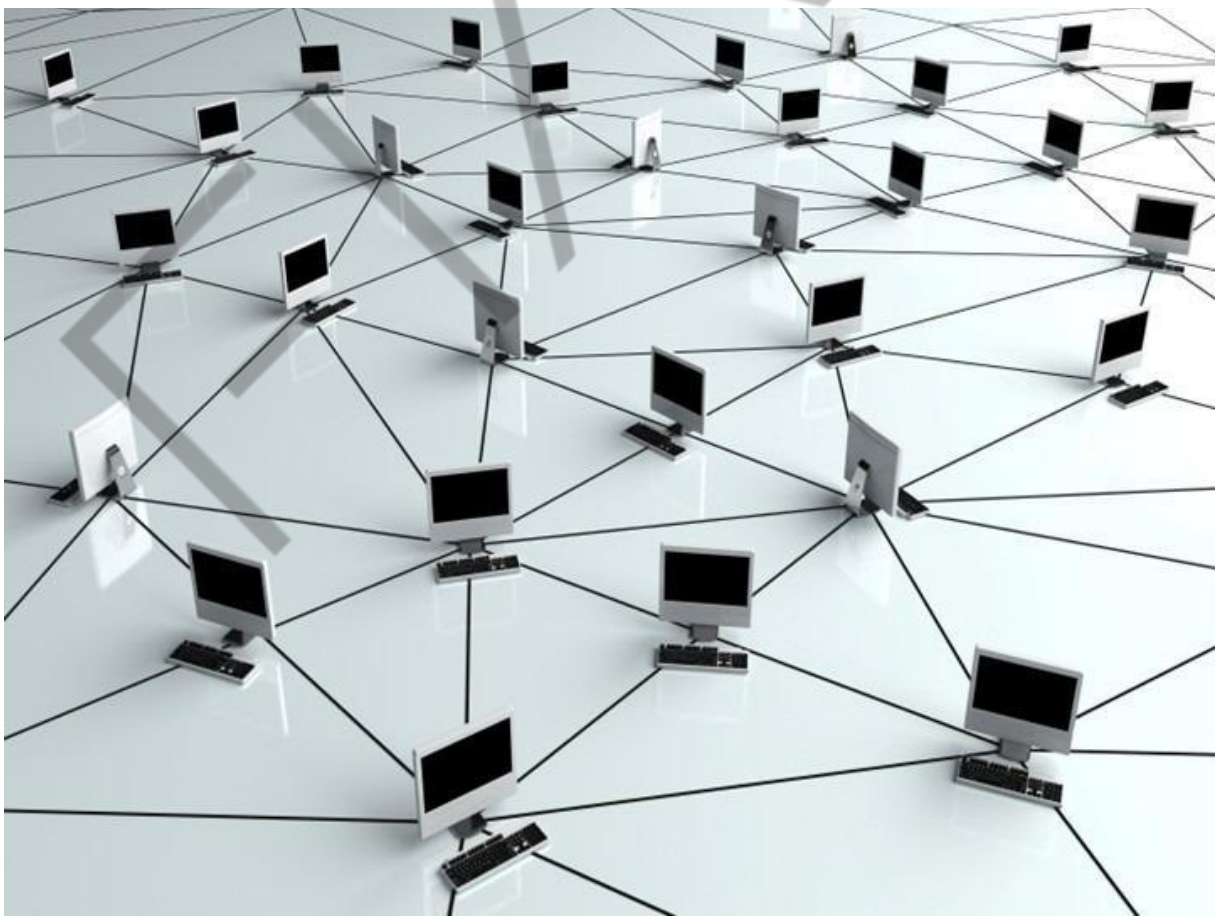


Figura 1 - O que é um sistema distribuído?
Fonte: pplware (2015)

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Os sistemas distribuídos são um conceito fundamental na computação moderna, permitindo que diferentes componentes de um sistema trabalhem em conjunto, mesmo as máquinas sendo distintas. Em vez de concentrar todo o processamento e dados em um único servidor, os sistemas distribuídos dividem suas tarefas entre vários computadores, garantindo maior eficiência e escalabilidade.

Um exemplo clássico é o da Netflix, que entrega filmes e séries para milhões de usuários ao mesmo tempo. Para que isso seja possível, os dados de um vídeo não estão em apenas um servidor; eles são replicados em servidores ao redor do mundo, garantindo que o conteúdo seja acessado com rapidez, mesmo em horários de pico.

Essa abordagem é essencial em um mundo conectado, onde serviços precisam atender milhões de usuários simultaneamente. Imagine um e-commerce que processa milhares de compras por minuto. Sem uma arquitetura distribuída, esse volume seria impossível de suportar.

Ao longo desta aula, exploraremos os principais conceitos e componentes que tornam os sistemas distribuídos tão poderosos, além de discutir os desafios que eles enfrentam e as soluções empregadas para superá-los.

Para entendermos melhor a importância dos sistemas distribuídos, vamos compará-los com as arquiteturas monolíticas. Em um sistema monolítico, todas as funcionalidades estão centralizadas em um único aplicativo. Isso pode parecer eficiente no início, mas rapidamente se torna um problema à medida que o sistema cresce.

Por exemplo, em um sistema de restaurante monolítico, o gerenciamento de pedidos, pagamentos e entregas está no mesmo código. Se o módulo de pagamentos tiver um problema, todo o sistema vai falhar. Em contrapartida, em uma arquitetura distribuída, cada funcionalidade é um serviço independente. Assim, o módulo de pedidos pode continuar funcionando, mesmo que o pagamento esteja com problemas.

Outro exemplo é o da Uber. Seus sistemas de localização, cálculo de tarifas e pagamento são separados, permitindo que eles sejam atualizados ou escalados de forma independente. Se a localização estiver lenta, o cálculo de tarifas e pagamentos não é afetado.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Essa separação traz flexibilidade e confiabilidade, mas também exige um esforço maior na implementação e gerenciamento, como veremos nos próximos tópicos.

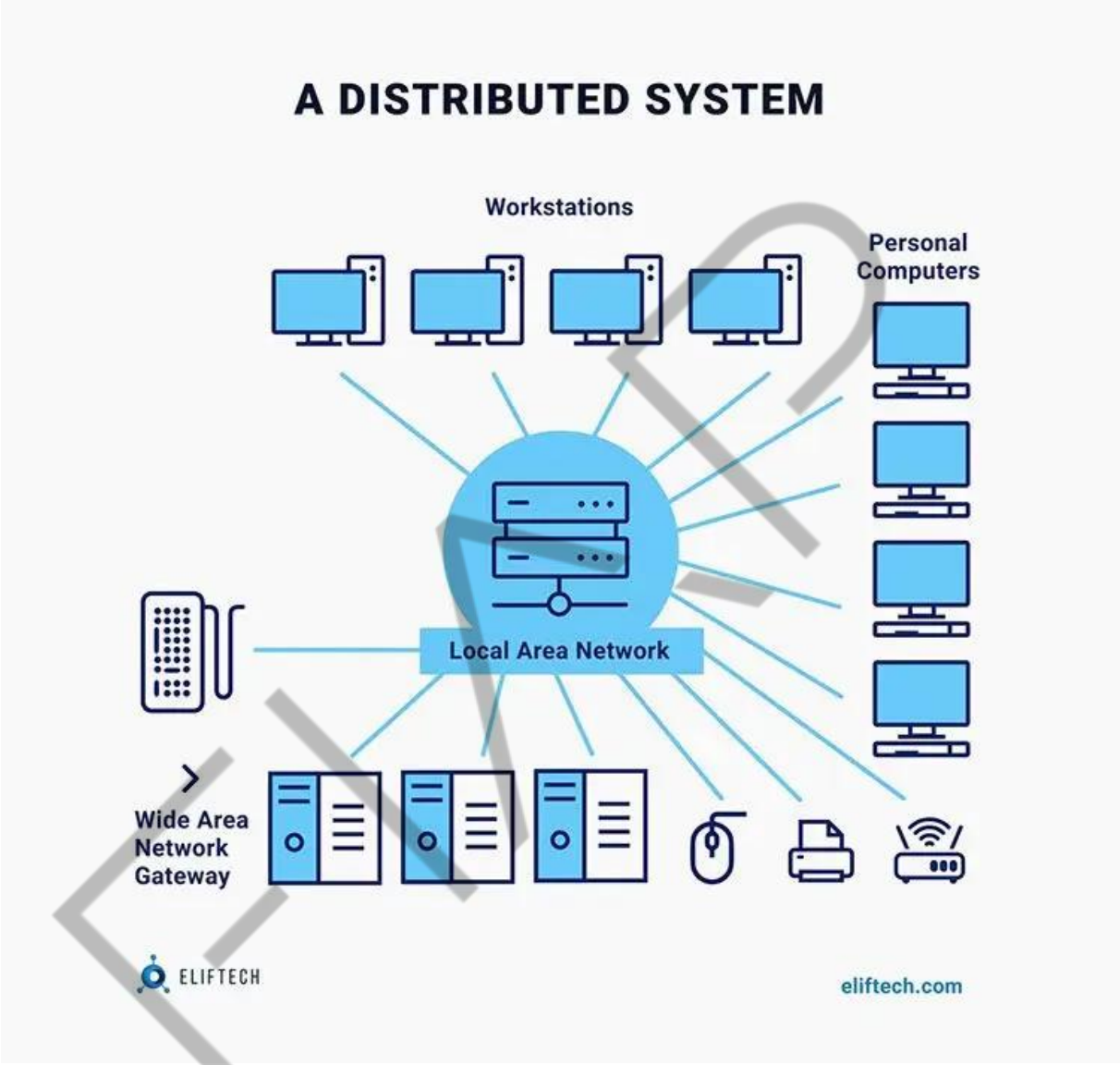


Figura 2 – A distributed system
Fonte: Eliftech (2022)

Como mencionado, Sistemas Distribuídos são redes compostas por computadores independentes que trabalham de forma integrada, apresentando-se como um sistema unificado. Esses sistemas compartilham recursos e coordenam tarefas entre diversos nós, permitindo que atuem coletivamente para alcançar objetivos comuns. Suas principais características incluem:

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

- **Múltiplos nós:** formados por vários computadores ou servidores interconectados que se comunicam por meio de uma rede.
- **Compartilhamento de recursos:** permitem o uso compartilhado de recursos, como poder de processamento, armazenamento e dados, entre os nós.
- **Escalabilidade:** podem crescer com a adição de novos nós, possibilitando o gerenciamento de maior carga ou a ampliação das funcionalidades.
- **Tolerância a falhas:** projetados para lidar com falhas de nós individuais sem comprometer o funcionamento geral do sistema.
- **Transparência:** buscam ocultar a complexidade da rede subjacente, apresentando-se aos usuários como uma única entidade coesa.

Nos sistemas distribuídos, um **nó** é qualquer dispositivo que participa do sistema, como servidores, dispositivos IoT ou mesmo o celular de um usuário. Esses nós interagem uns com os outros para executar tarefas complexas de forma colaborativa.

Por exemplo, em um aplicativo como o WhatsApp, seu celular é um cliente que envia mensagens para os servidores. Esses servidores, por sua vez, processam a mensagem e a enviam para o destinatário. Cada servidor é um nó no sistema.

Uma característica importante de nós é a capacidade de comunicação e sincronização. Em sistemas bancários, por exemplo, os nós devem garantir que os saldos das contas estejam sempre atualizados, mesmo que existam milhões de transações acontecendo ao mesmo tempo.

A falha de um nó pode causar problemas no sistema, mas, em arquiteturas bem projetadas, outros nós podem assumir as responsabilidades do nó falho, garantindo a continuidade do serviço.

A implementação de sistemas distribuídos apresenta diversos desafios importantes, entre os quais se destacam:

- **Heterogeneidade:** envolve lidar com diferenças entre sistemas operacionais, tipos variados de redes de comunicação e diferentes arquiteturas de hardware, exigindo soluções que integrem esses elementos de forma eficiente.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

- **Sistemas abertos:** para garantir que os sistemas possam se comunicar e trabalhar de maneira conjunta, é essencial que sejam abertos, permitindo maior flexibilidade.
- **Segurança:** um aspecto crucial para a troca de informações entre sistemas. Técnicas como a criptografia podem ser empregadas para garantir a confidencialidade e a integridade das mensagens transmitidas pela rede.
- **Escalabilidade:** sistemas distribuídos devem ser projetados para permitir a adição de novos nós de forma simples e econômica, sem comprometer a qualidade do serviço oferecido.
- **Tratamento de falhas:** como qualquer componente, seja um processo, computador ou rede, pode falhar de maneira isolada, o sistema deve ser resiliente e continuar funcionando normalmente, minimizando os impactos ao usuário final.
- **Concorrência:** em ambientes distribuídos com múltiplos usuários, é comum que recursos sejam acessados ou modificados simultaneamente, exigindo mecanismos para gerenciar esses acessos de forma segura e eficiente.
- **Qualidade de Serviço (QoS):** refere-se à capacidade do sistema de manter um serviço confiável e consistente, garantindo desempenho adequado mesmo sob diferentes condições de carga.

A comunicação entre nós é a espinha dorsal de um sistema distribuído. Para isso, são utilizados protocolos como HTTP/REST, GRPC ou sistemas de mensagens como RabbitMQ e Kafka. Esses protocolos definem como as informações são enviadas e recebidas entre nós.

Por exemplo, em sistemas de e-commerce, o front-end se comunica com o back-end por meio de APIs REST, solicitando dados de produtos ou processando pagamentos. Por outro lado, serviços internos podem usar mensagens assíncronas para gerenciar pedidos, garantindo maior resiliência.

Comparando as abordagens, enquanto o HTTP é ótimo para comunicações rápidas e diretas, sistemas como RabbitMQ são ideais para lidar com grandes volumes de dados de forma assíncrona, como o envio de notificações ou mensagens.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Uma parte essencial dos sistemas distribuídos é o armazenamento de dados. Em vez de concentrar tudo em um único banco de dados, os sistemas distribuídos utilizam bancos como MongoDB ou Cassandra, que replicam os dados entre diferentes servidores.

Por exemplo, em redes sociais, as postagens de um usuário precisam estar disponíveis rapidamente para seus amigos. Isso é possível porque os dados são distribuídos em vários servidores, reduzindo a latência de acesso.

Uma vantagem desse modelo é a alta disponibilidade. Mesmo que um servidor falhe, outros servidores podem continuar fornecendo os dados. No entanto, isso também traz desafios, como garantir a consistência dos dados em todos os servidores.

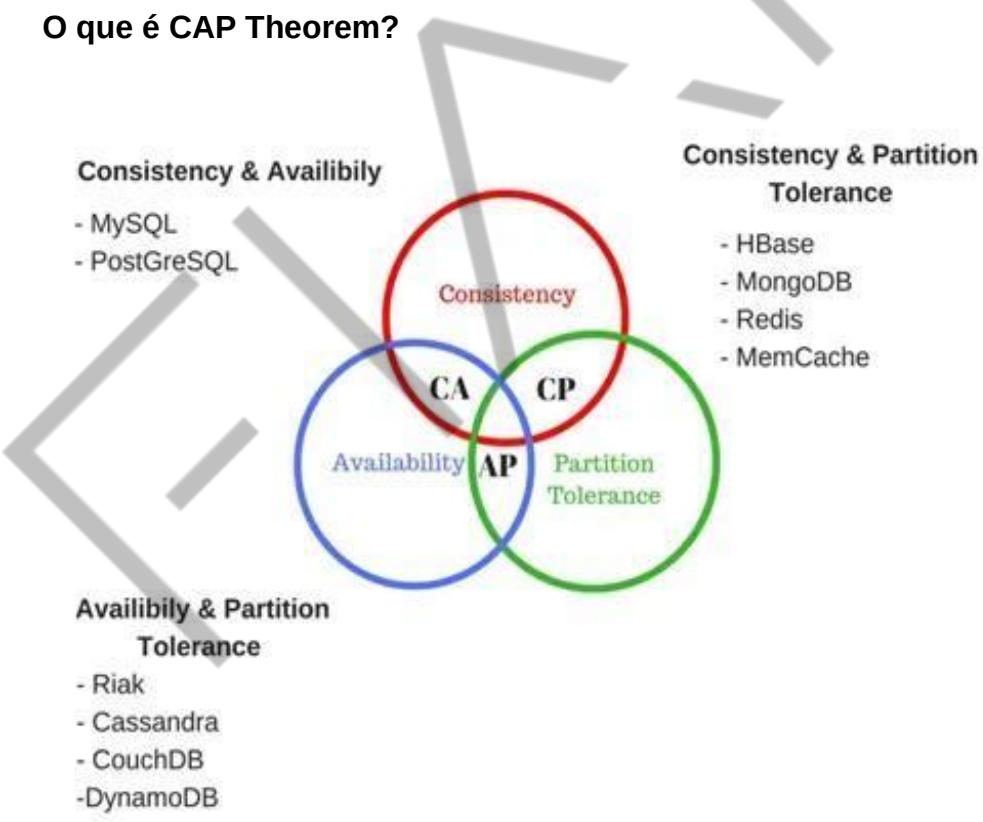


Figura 3 – CAP Theorem
Fonte: media.licn.com (2023)

O **CAP Theorem** estabelece que um sistema distribuído pode oferecer no máximo dois dos seguintes atributos ao mesmo tempo: consistência, disponibilidade e tolerância a partições.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

- **Consistência:** garante que todos os clientes vejam os mesmos dados, mesmo após atualizações em diferentes nós.
- **Disponibilidade:** assegura que o sistema esteja sempre acessível, mesmo diante de falhas.
- **Tolerância a partições:** permite que o sistema continue operando, mesmo em caso de falhas na conectividade entre os nós.

Por exemplo, bancos de dados como o MongoDB priorizam disponibilidade e tolerância a partições, permitindo que o sistema continue funcionando mesmo em caso de falhas de comunicação.

Já sistemas financeiros geralmente priorizam consistência, garantindo que os dados sejam sempre precisos, mesmo que isso implique em maior latência. Diferentes modelos de banco de dados NoSQL foram desenvolvidos para atender a essas demandas específicas de sistemas distribuídos:

- **Modelo orientado a documentos:** ideal para armazenar coleções de documentos semiestruturados, como JSON. Exemplos incluem MongoDB e CouchDB.
- **Modelo chave-valor:** focado em alto desempenho, com dados indexados por uma chave única, muito utilizado em aplicações como redes sociais e jogos. Exemplos incluem Redis e DynamoDB.
- **Modelo orientado a colunas:** armazena dados em colunas, oferecendo forte consistência, mas menor disponibilidade. Exemplos incluem Cassandra e HBase.
- **Modelo orientado a grafos:** especializado em armazenar dados com relações complexas, como conexões entre usuários em redes sociais. Exemplos incluem Neo4J e FlockDB.

Esses modelos destacam como diferentes abordagens podem ser adotadas para equilibrar as necessidades de escalabilidade, desempenho e confiabilidade em sistemas distribuídos.

Agora vamos explorar os estilos de arquiteturas distribuídas:

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Arquitetura Monolítica (não distribuído)

Uma aplicação única que abrange todas as funcionalidades, onde cada parte do sistema está interligada em um único código base. Exemplos: aplicativos tradicionais, como sistemas legados de gestão. Suas desvantagens são a dificuldade de escalar e manter e o fato de que uma falha pode derrubar todo o sistema. No entanto, é importante mencioná-la, pois muitos sistemas começam como monolíticos antes de migrarem para arquiteturas distribuídas.

Peer-to-Peer (P2P)

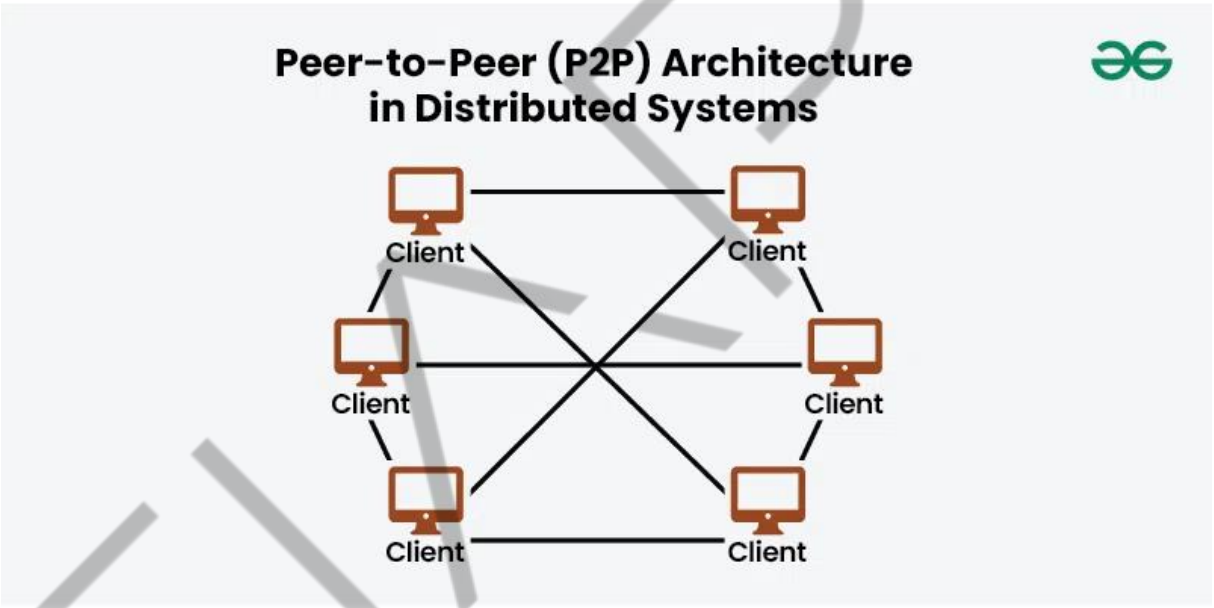


Figura 4 – Peer-to-Peer (P2P) Architecture
Fonte: geeksforgeeks.org (2024)

Imagine uma rede onde cada computador é tanto chefe quanto empregado. Ninguém “manda” em ninguém e todos colaboram para compartilhar arquivos, vídeos, músicas ou qualquer outra coisa que queiram. Essa é a ideia por trás da arquitetura P2P (peer-to-peer ou ponto a ponto, em português).

Em vez de ter um servidor central controlando tudo, cada computador (ou "par") da rede conversa diretamente com os outros. Um par pode pedir um arquivo para outro e assim por diante. A rede P2P é importante por sua resistência a falhas, economia, flexibilidade e caráter democrático, onde todos têm poder igual e podem contribuir. Porém, enfrenta desafios, como segurança contra vírus e hackers, desempenho variável e maior complexidade de organização.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Suas aplicações incluem compartilhamento de arquivos (como BitTorrent), criptomoedas (como Bitcoin) e jogos online, permitindo servidores descentralizados.

Data-Centric

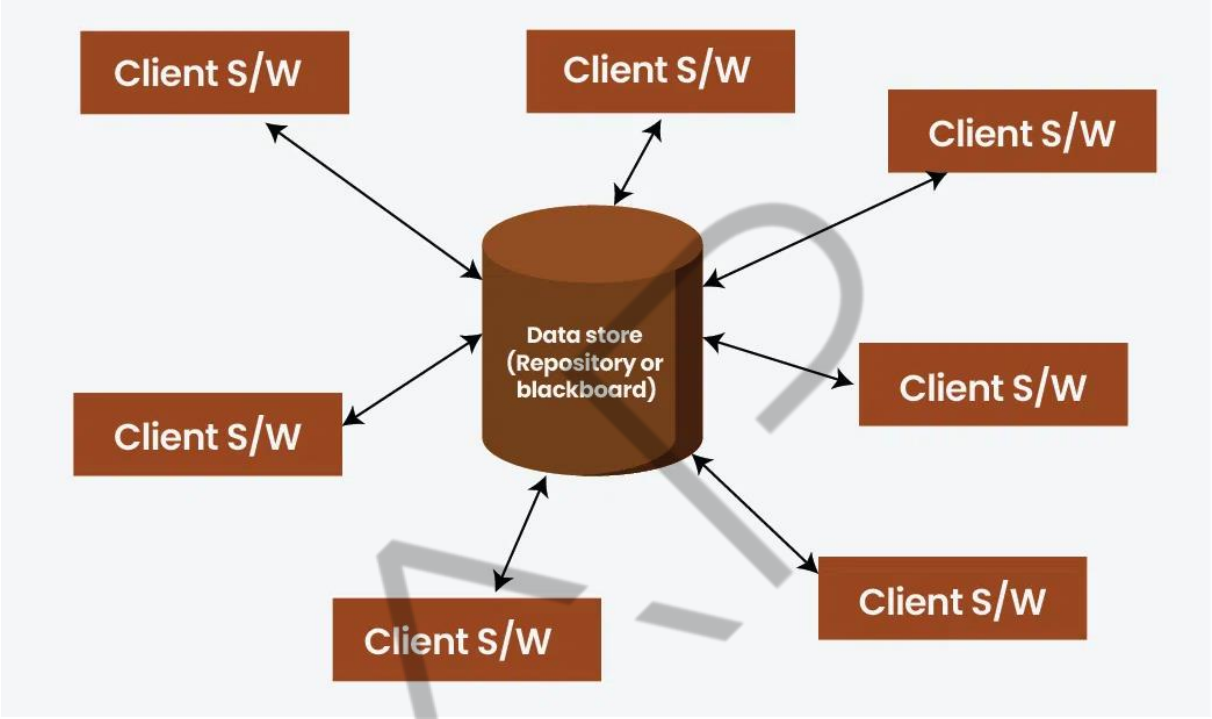


Figura 5 – Data-Centric Architecture
Fonte: geeksforgeeks.org (2024)

Imagine um sistema onde os dados são o foco principal. Todos os componentes do sistema giram em torno deles, acessando, armazenando e manipulando os dados de forma eficiente. Essa é a ideia por trás da Arquitetura Centrada em Dados (Data-Centric Architecture).

Ao invés de ter aplicações separadas controlando seus próprios dados, a Arquitetura Centrada em Dados reúne tudo em um só lugar: um repositório central. Vários serviços e aplicações acessam este repositório para obter, modificar e analisar os dados que precisam.

A Arquitetura Centrada em Dados é importante porque centraliza informações, garantindo consistência, facilita a integração de dados de várias fontes, melhora a qualidade dos dados ao evitar redundâncias e oferece eficiência no acesso e manipulação. Ela serve para gerenciar diversos tipos de informações (clientes, produtos, finanças), permitir análises complexas e integrar sistemas que

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

compartilhem os mesmos dados. Exemplos de soluções: banco de dados relacional, Data Warehouses e ERP.

Apesar das vantagens, é essencial avaliar se essa abordagem atende às necessidades específicas do sistema.

Service-Oriented Architecture (SOA)

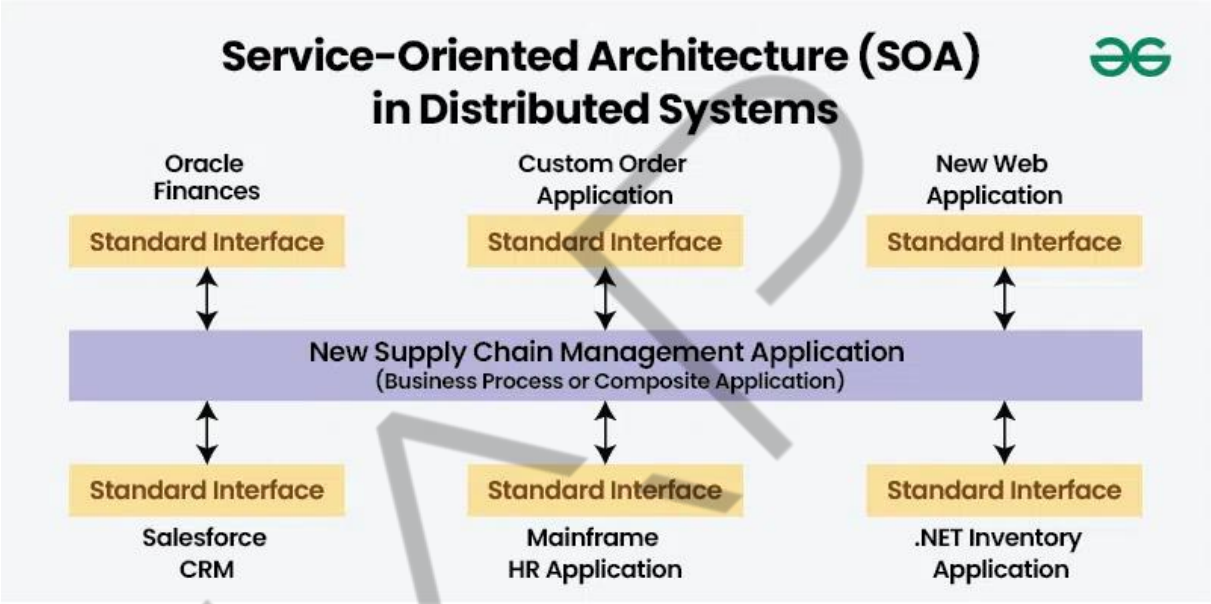


Figura 6 –Service-Oriented Architecture (SOA)
Fonte: geeksforgeeks.org (2024)

A Arquitetura Orientada a Serviços (SOA) é um modelo de design onde as funcionalidades do sistema são organizadas como serviços independentes que se comunicam por meio de uma rede. Cada serviço realiza uma tarefa específica, como autenticação, processamento de pagamentos ou envio de notificações, e pode ser utilizado por diferentes aplicações. Um elemento importante na SOA é o uso de um Barramento de Serviços Empresarial (ESB - Enterprise Service Bus), que facilita a comunicação e integração entre os serviços.

Na SOA, os serviços são projetados para serem independentes e interoperáveis, comunicando-se por meio de protocolos padronizados como HTTP, SOAP ou REST. O ESB atua como uma camada intermediária que conecta os serviços, gerenciando o fluxo de dados entre eles. Ele simplifica a integração ao oferecer funcionalidades como roteamento de mensagens, transformação de dados e orquestração de processos, garantindo que diferentes serviços possam se comunicar,

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

mesmo que usem tecnologias distintas. Com isso, o ESB reduz a complexidade e centraliza o gerenciamento da comunicação entre os serviços.

Imagine uma loja virtual que precisa integrar serviços como cálculo de frete, processamento de pagamentos e gestão de estoque. Cada serviço pode estar em sistemas separados, usando tecnologias diferentes. O ESB entra em cena para intermediar essa comunicação, roteando as solicitações do sistema de pedidos para os serviços corretos.

Event-Driven Architecture

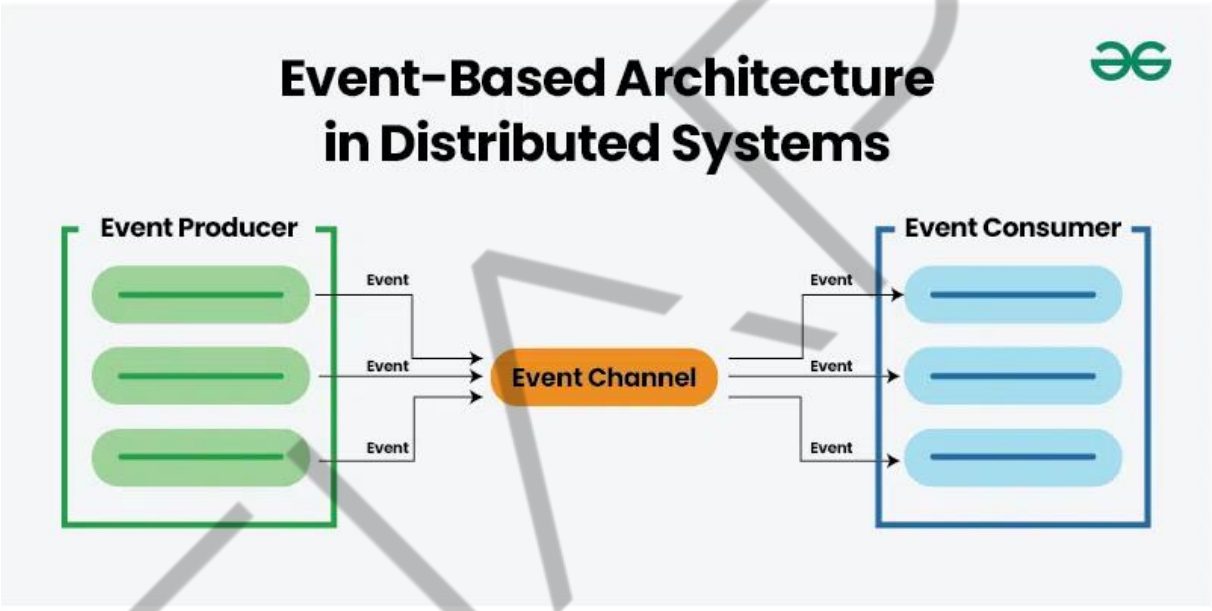


Figura 7 – Event-Based Architecture
Fonte: geeksforgeeks.org (2024)

A Arquitetura Orientada a Eventos (EDA) é um padrão arquitetural em que o fluxo de dados e o controle de um sistema são determinados por eventos. Os componentes em um sistema EDA se comunicam produzindo e consumindo eventos, que representam mudanças de estado ou ações dentro do sistema.

Na EDA, os componentes são divididos em produtores e consumidores de eventos. Os produtores são responsáveis por gerar eventos para sinalizar alterações de estado ou ações específicas. Esses eventos são transmitidos para os consumidores por meio de canais de eventos, como filas de mensagens ou fluxos de eventos.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Os consumidores escutam e processam os eventos conforme necessário. Essa abordagem promove baixo acoplamento, pois produtores e consumidores interagem indiretamente por meio de eventos, em vez de chamadas diretas, o que torna o sistema mais flexível e adaptável.

Um exemplo comum de EDA é uma plataforma de análise em tempo real, como sistemas de negociação de ações, que processam e reagem a eventos do mercado à medida que eles ocorrem.

Client-Server

A Arquitetura Cliente-Servidor é um modelo fundamental em sistemas distribuídos, onde o sistema é dividido em dois componentes principais: clientes e servidores. Essa arquitetura define como tarefas e serviços são distribuídos entre diferentes entidades em uma rede, permitindo uma interação estruturada e eficiente.

Na arquitetura cliente-servidor, os clientes são responsáveis por solicitar serviços ou recursos, enquanto os servidores fornecem esses serviços. O cliente inicia uma solicitação, o servidor processa e retorna a resposta correspondente. Essa abordagem centraliza a gestão de recursos e serviços no servidor, enquanto o cliente foca na interação com o usuário e na apresentação de informações. A comunicação segue um modelo de requisição e resposta, utilizando protocolos bem definidos, como HTTP.

Um exemplo clássico é uma aplicação web, onde navegadores (clientes) solicitam páginas ou dados de um servidor web. Outro caso comum são os sistemas de e-mail, onde clientes de e-mail, como Outlook ou Gmail, se conectam a servidores para enviar, receber e gerenciar mensagens.

Microservices

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

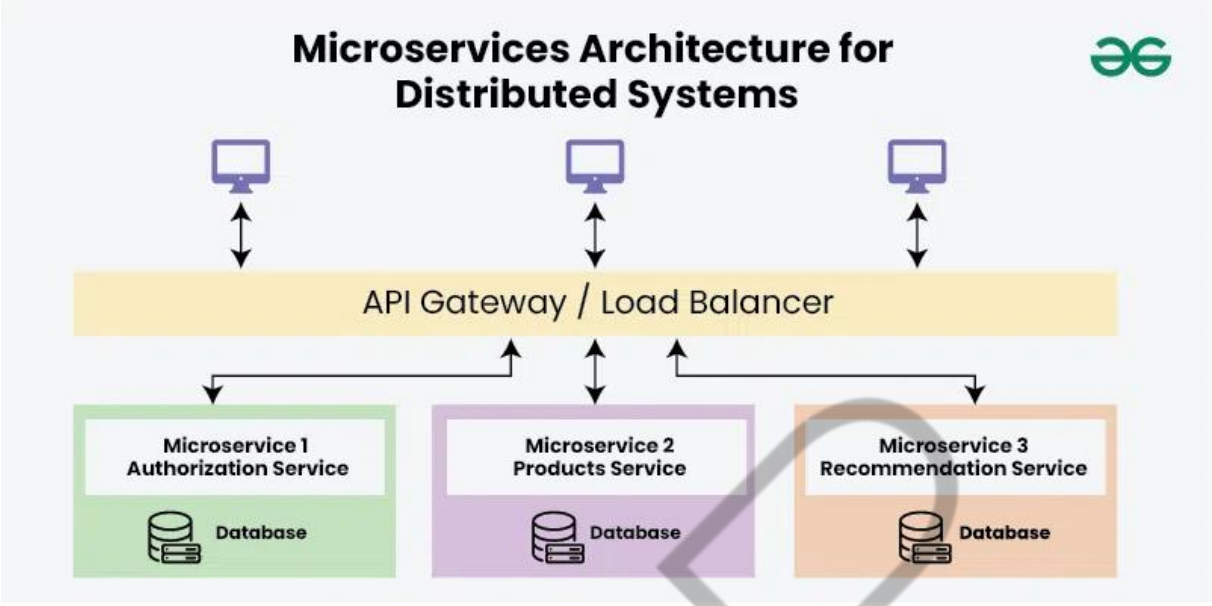


Figura 8 – Microservices
Fonte: geeksforgeeks.org (2024)

A Arquitetura de Microsserviços é um padrão de design onde uma aplicação é composta por pequenos serviços independentes, cada um responsável por uma funcionalidade específica. Esses serviços são fracamente acoplados e se comunicam por meio de protocolos leves, como HTTP ou filas de mensagens.

Na arquitetura de microsserviços, cada serviço é projetado para realizar uma única tarefa ou função de negócio, promovendo modularidade e organização. Esses serviços são autônomos, o que significa que podem ser desenvolvidos, implantados e escalados de forma independente, sem interferir nos outros.

Cada microsserviço gerencia seu próprio banco de dados, evitando dependências entre serviços e melhorando a escalabilidade. A comunicação entre os serviços ocorre por APIs bem definidas ou por protocolos de mensagens, garantindo integração sem acoplamento direto.

Um exemplo comum é o de plataformas de e-commerce, como a Amazon, que utiliza microsserviços para lidar com diferentes aspectos do negócio, como autenticação de usuários, processamento de pagamentos e gestão de pedidos.

Os sistemas distribuídos, apesar de extremamente poderosos, enfrentam desafios inerentes à sua complexidade e dependência de múltiplos componentes.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Aqui temos os principais desafios, além de soluções práticas e exemplos reais que ajudam a entender como superá-los:

Latência: atrasos na comunicação entre nós

A distância física entre os nós e a quantidade de dados transmitidos podem causar atrasos, afetando diretamente a experiência do usuário. Um exemplo prático em streaming de vídeo, como o da Netflix, é que atrasos na entrega de dados resultam em buffering constante. Soluções:

CDNs (Content Delivery Networks): servidores espalhados pelo mundo que armazenam dados próximos aos usuários, reduzindo a latência. **Exemplo:** Netflix utiliza CDNs para entregar vídeos de servidores regionais, garantindo carregamento rápido.

Caching: dados frequentemente acessados são armazenados em cache nos servidores para reduzir o tempo de acesso. **Exemplo:** Amazon utiliza o Redis como cache para acelerar consultas ao catálogo de produtos.

Consistência: garantir dados iguais em todos os nós

Manter os dados sincronizados em todos os nós de um sistema distribuído, especialmente em larga escala, é complexo. Um exemplo prático é um banco: o saldo de uma conta precisa estar atualizado em todas as agências e dispositivos. Soluções:

Replicação eventual: bancos de dados distribuídos, como o MongoDB, sincronizam os dados entre nós de forma assíncrona, garantindo consistência eventual.

Benefício: alta disponibilidade, mesmo que os dados levem um tempo para sincronizar.

Replicação síncrona: bancos como o PostgreSQL podem replicar dados em tempo real, garantindo consistência total, mas com impacto na latência.

Cuidado: útil em sistemas críticos, mas deve ser usado com parcimônia em cenários de alta escalabilidade.

Resiliência: lidar com falhas de componentes

Em sistemas distribuídos, falhas são inevitáveis. Um nó pode ficar inativo, mas o sistema precisa continuar operando sem interrupções. Um exemplo prático: se um

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

serviço de pagamento falhar em um e-commerce, o carrinho e a pesquisa de produtos devem continuar funcionando. Soluções:

Circuit Breaker: monitora os serviços e bloqueia requisições a serviços com falhas para evitar sobrecarga.

Ferramentas: Resilience4j, Hystrix.

Exemplo Prático: se o gateway de pagamento estiver indisponível, o Circuit Breaker ativa um fallback para salvar os pedidos como "pendentes".

Retry com Backoff: reenvia requisições falhas após intervalos crescentes, reduzindo o impacto de falhas temporárias.

Exemplo Prático: em sistemas bancários, o retry garante que uma transferência seja processada em caso de falhas na rede.

Bulkhead: isola recursos críticos, evitando que falhas em uma parte do sistema afetem todo o serviço.

Exemplo Prático: em um sistema de reservas de hotel, cada funcionalidade (ex.: reservas, pagamentos) usa recursos dedicados para evitar impacto global.

Segurança: garantir comunicação segura

Sistemas distribuídos envolvem múltiplos nós, aumentando os pontos de vulnerabilidade para ataques e intercepções de dados. Um exemplo prático é o de redes financeiras, como as de cartões de crédito, onde interceptações podem levar a fraudes. Solução:

Criptografia: a comunicação entre nós deve ser criptografada usando HTTPS ou protocolos seguros, como TLS.

Autenticação e autorização: ferramentas como OAuth e JWT garantem que apenas usuários e serviços autorizados acessem os dados.

Monitoramento e diagnóstico

Monitorar um sistema distribuído com múltiplos componentes é complexo, mas essencial para identificar e resolver problemas rapidamente. Em um e-commerce, um serviço de logística pode estar funcionando lentamente sem que os outros componentes percebam. Ferramentas como Prometheus e Grafana consolidam métricas e logs de todos os componentes.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Exemplo: um dashboard no Grafana monitora latência, taxa de erros e tráfego entre microsserviços.

Tracing Distribuído: ferramentas como Jaeger e Zipkin ajudam a rastrear chamadas entre serviços, identificando gargalos ou falhas.

Nessa aula de arquitetura distribuída, entendemos como escalabilidade e eficiência sustentam serviços como os da Netflix e Amazon. Garantir que esses sistemas não falhem mesmo sob pressão é a próxima peça do quebra-cabeça. Na próxima aula, exploraremos Padrões de Resiliência, entendendo como sistemas se protegem contra falhas usando estratégias como Circuit Breaker e Retry. Você vai aprender como criar sistemas prontos para enfrentar o inesperado.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nessa aula, exploramos a arquitetura distribuída e sua importância em sistemas do nosso dia a dia, além dos tipos de arquiteturas de sistemas distribuídos, com alguns exemplos e estilos de arquitetura. Também falamos sobre os desafios de cada arquitetura.

Entendemos os fundamentos que tornam sistemas como os da Netflix e da Amazon tão confiáveis e eficientes. Isso será a base para o seu próximo passo, o de construir sistemas resilientes.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

AWS. **CAP Theorem.** Disponível em: https://docs.aws.amazon.com/pt_br/whitepapers/latest/availability-and-beyond-improving-resilience/cap-theorem.html. Acesso em: 12 dez . 2024.

ELIFTECH. **Distributed Architecture: How can it benefit your payment system? 2023**. Disponível em: https://www.eliftech.com/insights/distributed-architecture-how-can-it-benefit-your-payment-system/. Acesso em: 12 dez. 2024.

FOWLER, M. **Patterns of Enterprise Application Architecture**. Boston: Addison-Wesley, 2003.

GEEKSFORGEEKS. **ARCHITECTURE STYLES in distributed systems**. 2023. Disponível em: https://www.geeksforgeeks.org/architecture-styles-in-distributed-systems/. Acesso em: 12 dez. 2024.

PPLWARE. **Afinal, o que é um sistema distribuído?** 2023. Disponível em: https://pplware.sapo.pt/informacao/afinal-o-que-e-um-sistema-distribuido/. Acesso em: 12 dez. 2024.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

Resiliência. Escalabilidade Distribuídos.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com